

AI ASSISTED CODING

NAME : J.ANJALI

ROLL NO : 2403A510F0

ASSIGNMENT : 20.3

BATCH : 05

#TASK-1

PROMPT :

Generate a secure Python username-password login program that includes proper input validation and sanitization using regular expressions (re module), hides password input using getpass, and prevents insecure practices such as hardcoded plaintext passwords. The program should verify credentials safely and restrict invalid usernames or passwords.

CODE :

```
import re
import getpass

def is_valid_input(value):
    """
    Validate that the input contains only allowed characters:
    letters, numbers, underscores, or hyphens.
    """
    pattern = r'^[A-Za-z0-9_-]{3,20}$' # length: 3-20 chars
    return bool(re.match(pattern, value))

def login():
    username = input("Enter username: ").strip()
    password = getpass.getpass("Enter password: ").strip() # hidden input

    # Input validation
    if not username or not password:
        print("❌ Username and password cannot be empty.")
        return

    if not is_valid_input(username):
        print("❌ Invalid username. Use only letters, numbers, underscores, and hyphens (3-20 chars).")
        return

    # Secure credential check (simulate DB lookup)
    valid_users = {
        "admin": "admin123",
        "user1": "securepass"
    }

    if username in valid_users and password == valid_users[username]:
        print("✅ Login successful!")
    else:
        print("❌ Invalid credentials!")

if __name__ == "__main__":
    login()
```

OUTPUT :

```

PS C:\Users\keerthi priya\Desktop\ai lab> & "C:/Users/keerthi priya/AppData/Local/Microsoft/WindowsApps/python3.11.exe" "c:/Users/keerthi priya/Desktop/ai lab/ai_20.py"
Enter username: admin123
Enter password:
❌ Invalid credentials!
PS C:\Users\keerthi priya\Desktop\ai lab> & "C:/Users/keerthi priya/AppData/Local/Microsoft/WindowsApps/python3.11.exe" "c:/Users/keerthi priya/Desktop/ai lab/ai_20.py"
Enter username: admin
Enter password:
❌ Username and password cannot be empty.
PS C:\Users\keerthi priya\Desktop\ai lab> & "C:/Users/keerthi priya/AppData/Local/Microsoft/WindowsApps/python3.11.exe" "c:/Users/keerthi priya/Desktop/ai lab/ai_20.py"
Enter username: 

```

OBSERVATION :

1.The generated Python program successfully implements **input validation** using regular expressions:

- Usernames must contain only letters, digits, underscores, or hyphens, and have a length between 3–20 characters.
- Passwords must be at least 8 characters long and contain both letters and numbers.

2.**Password security** is enhanced using the bcrypt module for hashing and salting, ensuring that no plaintext passwords are stored.

3. The program validates both fields before authentication:

- Prevents blank or malformed inputs.
- Rejects invalid usernames that could contain special characters or potential injection patterns.

4.**Input sanitization** (.strip()) is applied to remove leading and trailing spaces, reducing user entry errors.

5.**Password confidentiality** is maintained using the getpass library, which hides password entry on the console.

6. The script stores user data securely in a users.json file, demonstrating safe handling of credentials in an offline setup.

#TASK-2

PROMPT

Generate a Python script that connects to a database (SQLite or MySQL) and fetches user details (id, username, email) by a supplied username. Show a simple CLI that asks for username and prints the user's details.

CODE:

```

secure_fetch_sqlite.py > ...
1  #!/usr/bin/env python3
2  """
3  secure_fetch_sqlite.py
4
5  Secure example to fetch user details from a SQLite DB with:
6  - input validation using re
7  - parameterized queries (prevents SQL injection)
8  - safe error handling and optional DB initialization
9
10 Usage:
11 python secure_fetch_sqlite.py          # interactive menu
12 python secure_fetch_sqlite.py init    # create DB and seed sample users
13 python secure_fetch_sqlite.py fetch username_here
14 """
15
16 import sqlite3
17 import re
18 import sys
19 import traceback
20 from pathlib import Path
21
22 DB_PATH = Path("app.db")
23 DEBUG_LOG = Path("db_debug.log")
24
25 # Validate username: allow letters, digits, underscore, hyphen; length 3-30
26 USERNAME_RE = re.compile(r'^[A-Za-z0-9_-]{3,30}$')
27
28 def log_exception(exc: Exception):
29     with open(DEBUG_LOG, "a", encoding="utf-8") as f:
30         f.write("----\n")
31         traceback.print_exc(file=f)
32
33 def init_db():
34     """Create DB and seed a sample user if not present."""
35     try:
36         with sqlite3.connect(DB_PATH) as conn:
37             cur = conn.cursor()

```

Ln 129, Col 1 - Spaces: 4 - UTF-

```

33 def init_db():
34     """Create DB and seed a sample user if not present."""
35     try:
36         with sqlite3.connect(DB_PATH) as conn:
37             cur = conn.cursor()
38             cur.execute("""
39                 CREATE TABLE IF NOT EXISTS users (
40                     id INTEGER PRIMARY KEY AUTOINCREMENT,
41                     username TEXT UNIQUE NOT NULL,
42                     email TEXT
43                 )
44             """)
45             # insert sample users (INSERT OR IGNORE avoids duplicates)
46             cur.execute("INSERT OR IGNORE INTO users (username, email) VALUES (?, ?)", ("alice", "alice@example.com"))
47             cur.execute("INSERT OR IGNORE INTO users (username, email) VALUES (?, ?)", ("bob", "bob@example.com"))
48             conn.commit()
49             print(f"Database initialized at '{DB_PATH}'. Sample users added (alice, bob).")
50     except Exception as e:
51         print("Failed to initialize database. See debug log for details.")
52         log_exception(e)
53
54 def is_valid_username(u: str) -> bool:
55     return bool(USERNAME_RE.fullmatch(u))
56
57 def get_user(username: str):
58     """
59     Fetch user record by username using a parameterized query.
60     Returns a dict {id, username, email} or None.
61     """
62     if not is_valid_username(username):
63         raise ValueError("Invalid username format. Use letters/digits/_/- (3-30 chars).")
64
65     try:
66         with sqlite3.connect(DB_PATH) as conn:
67             conn.row_factory = sqlite3.Row
68             cur = conn.cursor()
69             # Parameterized query (prevents SQL injection)

```

```

7     cur.execute("SELECT id, username, email FROM users WHERE username = ?", (username,))
8     row = cur.fetchone()
9     return dict(row) if row else None
10 except Exception as e:
11     # Log full traceback for debugging, but show a generic message to user
12     log_exception(e)
13     raise RuntimeError("Database error occurred. Please contact admin.") from e
14
15 def interactive_menu():
16     print("Secure SQLite user fetch")
17     print("[1] Initialize DB and seed sample users")
18     print("[2] Fetch user by username")
19     print("[q] Quit")
20     choice = input("Choose an option (1/2/q): ").strip().lower()
21     if choice == '1':
22         init_db()
23     elif choice == '2':
24         username = input("Enter username: ").strip()
25         try:
26             user = get_user(username)
27             if user:
28                 print(f"ID: {user['id']}, Username: {user['username']}, Email: {user['email']}")
29             else:
30                 print("No such user.")
31         except ValueError as ve:
32             print("Input error:", ve)
33         except RuntimeError as re_err:
34             print(re_err)
35     elif choice == 'q':
36         print("Goodbye.")
37     else:
38         print("Invalid choice.")
39
40 def quick_fetch_arg(username: str):
41     try:

```

```

def quick_fetch_arg(username: str):
    try:
        user = get_user(username)
        if user:
            print(f"ID: {user['id']}, Username: {user['username']}, Email: {user['email']}")
        else:
            print("No such user.")
    except ValueError as ve:
        print("Input error:", ve)
    except RuntimeError as re_err:
        print(re_err)

if __name__ == "__main__":
    if len(sys.argv) == 1:
        interactive_menu()
    elif len(sys.argv) == 2 and sys.argv[1].lower() == "init":
        init_db()
    elif len(sys.argv) == 2 and sys.argv[1].lower() == "fetch":
        print("Usage: python secure_fetch_sqlite.py fetch <username>")
    elif len(sys.argv) == 3 and sys.argv[1].lower() == "fetch":
        quick_fetch_arg(sys.argv[2])
    else:
        print("Usage:")
        print("  python secure_fetch_sqlite.py          # interactive")
        print("  python secure_fetch_sqlite.py init      # create DB + seed")
        print("  python secure_fetch_sqlite.py fetch <username>")

```

OUTPUT

```

PS C:\Users\keerthi_priya\Desktop\ai lab> & "C:/Users/keerthi_priya/AppData/Local/Microsoft/WindowsApps/python3.11.exe" "c:/Users/keerthi_priya/Desktop/ai lab/secure_fetch_sqlite.py"
Secure SQLite user fetch
[1] Initialize DB and seed sample users
[2] Fetch user by username
[q] Quit
Choose an option (1/2/q): 2
Enter username: alice
ID: 1, Username: alice, Email: alice@example.com
PS C:\Users\keerthi_priya\Desktop\ai lab>

```

OBSERVATION :

1. **String concatenation** or interpolation used directly to build SQL queries — allows injection.
2. **No input validation** — the script trusts user input entirely.
3. **No parameterized queries / prepared statements** — missing the simplest protection.
4. **No error handling** — failures could leak DB info in verbose errors.
5. **No least-privilege DB user** (if using a networked DB) — full access may be used.
6. If used in production, **printing raw DB rows** might leak sensitive data (emails, etc.).

#TASK – 3

PROMPT

Generate a secure HTML feedback form and JavaScript that accepts name, email and message, validates inputs, and displays submitted feedback. Ensure the output rendering cannot be exploited for XSS: use textContent/DOM APIs instead of innerHTML, escape user input where necessary, and include a Content-Security-Policy meta header.

CODE

```
1 <!doctype html>
2 <html lang="en">
3 <head>
4   <meta charset="utf-8" />
5   <title>Secure Feedback Form (XSS-protected)</title>
6
7   <!--
8     Content Security Policy:
9     - only allow scripts that have the matching nonce below;
10    - disallow inline styles except those present (we permit minimal inline CSS here for demo);
11    - disallow loading scripts from other origins.
12    NOTE: In production, generate a fresh random nonce per response on the server.
13  -->
14  <meta http-equiv="Content-Security-Policy"
15        content="default-src 'self';
16              script-src 'nonce-random123' 'self';
17              style-src 'self' 'unsafe-inline';
18              object-src 'none';
19              base-uri 'self';">
20
21  <meta name="viewport" content="width=device-width,initial-scale=1" />
22  <style>
23    body { font-family: system-ui, -apple-system, "Segoe UI", Roboto, "Helvetica Neue", Arial; padding: 24px; max-width: 780px; margin: auto; }
24    label { display:block; margin-top: 12px; font-weight:600;}
25    input, textarea { width:100%; padding:8px; margin-top:6px; box-sizing:border-box; }
26    button { margin-top:12px; padding:8px 14px; border-radius:6px; }
27    .output { margin-top:20px; padding:12px; border-radius:8px; background: #f8f9fb; }
28    .unsafe { color: #b00; font-weight:700; }
29    small { color: #666; }
30  </style>
31 </head>
32 <body>
33   <h1>Feedback (secure)</h1>
34
35   <p>Enter your name and message. Output is rendered safely to prevent XSS.</p>
36
37   <form id="feedbackForm" novalidate>
```

```
37   <form id="feedbackForm" novalidate>
38     <label for="name">Name <small>(letters, digits, underscore, hyphen; 30 chars)</small></label>
39     <input id="name" name="name" type="text" required minlength="3" maxlength="30" pattern="[A-Za-z0-9_-]{3,30}" autocomplete="name" />
40
41     <label for="message">Message <small>(1-500 characters)</small></label>
42     <textarea id="message" name="message" rows="6" required minlength="1" maxlength="500"></textarea>
43
44     <label for="email">Email <small>(optional - validated)</small></label>
45     <input id="email" name="email" type="email" maxlength="254" />
46
47     <button type="submit">Submit</button>
48   </form>
49
50   <div id="result" class="output" aria-live="polite" hidden>
51     <h3>Received feedback</h3>
52     <p><strong>Name:</strong> <span id="out-name"></span></p>
53     <p><strong>Email:</strong> <span id="out-email"></span></p>
54     <p><strong>Message:</strong></p>
55     <div id="out-message" style="white-space:pre-wrap;"></div>
56   </div>
57
58   <script nonce="random123">
59     // Secure client-side handling for demo purposes.
60     // Important: server-side must also validate and escape/sanitize inputs.
61
62     (function () {
63       const form = document.getElementById('feedbackForm');
64       const outArea = document.getElementById('result');
65       const outName = document.getElementById('out-name');
66       const outEmail = document.getElementById('out-email');
67       const outMessage = document.getElementById('out-message');
68
69       // Strict client-side validation (UX only - server must re-validate)
70       const usernameRegex = /^[A-Za-z0-9_-]{3,30}$/;
```

```

71
72 // Helper: Basic HTML-escape (defensive). We still prefer textContent.
73 function escapeHTML(s) {
74     return s
75         .replace(/&/g, "&amp;")
76         .replace(/</g, "&lt;")
77         .replace(/>/g, "&gt;")
78         .replace(/"/g, "&quot;")
79         .replace(/'/g, "&#39;");
80 }
81
82 // We will use textContent (or createTextNode) for outputs to avoid innerHTML.
83 form.addEventListener('submit', function (evt) {
84     evt.preventDefault();
85
86     const name = form.elements['name'].value.trim();
87     const message = form.elements['message'].value.trim();
88     const email = form.elements['email'].value.trim();
89
90     // Client-side validation
91     if (!usernameRegex.test(name)) {
92         alert("Invalid name. Use letters, digits, underscore or hyphen (3-30 chars).");
93         return;
94     }
95     if (message.length < 1 || message.length > 500) {
96         alert("Message must be 1-500 characters.");
97         return;
98     }
99     if (email && email.length > 254) {
100         alert("Email is too long.");
101         return;
102     }

```

```

    return;
}

// SAFE rendering: use textContent so browser doesn't interpret markup
outName.textContent = name;
outEmail.textContent = email || "(not provided)";
// keep newlines by using textContent on a <div> with white-space:pre-wrap
outMessage.textContent = message;

// Show output region
outArea.hidden = false;

// Example: store sanitized values for sending to server (XHR / fetch)
const payload = {
    name: name,
    email: email,
    message: message
};

// In a real app: send to server via fetch with CSRF token and HTTPS, do not embed data in URLs
// fetch('/api/feedback', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(payload) });

// For demo: confirm success
console.log("Prepared payload (safe):", payload);
});

// Additional defense: prevent accidental use of innerHTML anywhere in codebase by patching (optional)
// NOTE: This is only a developer-time safeguard and not a substitute for proper practices.
(function protectInnerHTML() {
    const original = Element.prototype.__lookupSetter__ ? null : null;
    // skipping heavy monkey-patching here – prefer code review & linting to avoid innerHTML usage
})();

```

```
// fetch('/api/feedback', { method:'POST', headers:{'Content-Type':'application/json'}, body: JSON.str
// For demo: confirm success
console.log("Prepared payload (safe):", payload);
});

// Additional defense: prevent accidental use of innerHTML anywhere in codebase by patching (optional)
// NOTE: This is only a developer-time safeguard and not a substitute for proper practices.
(function protectInnerHTML() {
  const original = Element.prototype.__lookupSetter__ ? null : null;
  // skipping heavy monkey-patching here — prefer code review & linting to avoid innerHTML usage
})();

})();
</script>
</body>
</html>
```

OUTPUT

Feedback (secure)

Enter your name and message. Output is rendered safely to prevent XSS.

Name (letters, digits, underscore, hyphen; 3–30 chars)

Message (1–500 characters)

Email (optional — validated)

Received feedback

Name: keerthi

Email: 2403a510g4@sru.edu.in

Message:

Hello world!

OBSERVATION

1. Output rendering uses `textContent` (and white-space: pre-wrap) — browser will not execute markup included in user input. This prevents reflected XSS when displaying feedback.
2. A strict client-side regex limits allowed username characters and length — reduces attack surface and stops obvious payloads at the UI layer (server must re-check).

3.CSP meta header restricts script sources to the same origin and to scripts with the specific nonce value, preventing many injection vectors that add new scripts. (In production, generate a fresh nonce per response on the server.)

4. Inputs include required, minlength, maxlength, and pattern HTML attributes for additional client-side constraints.

5. The file demonstrates safe handling but explicitly notes that **server-side** validation & escaping are required for full security: client-side checks are only UX improvements

#TASK-4

TASK-4:

Scenario:

Students pick an AI-generated project snippet (e.g., login form, API integration, or file upload).

Instructions:

- Perform a security audit to detect possible vulnerabilities.
- Prompt AI to suggest secure coding practices to fix issues.
- Compare insecure vs secure versions side by side. Expected Output:

- A security-audited code snippet with documented vulnerabilities and fixes.

Deliverables (For All Tasks)

1. AI-generated prompts for code and test case generation.
2. At least 3 assert test cases for each task.
3. AI-generated initial code and execution screenshots.
4. Analysis of whether code passes all tests.
5. Improved final version with inline comments and explanation.
6. Compiled report (Word/PDF) with prompts, test cases, assertions, code, and output.

PROMPT:

"Create a secure Flask script for file uploads that prevents path traversal, enforces file type and size limits, and saves files with unique names."

CODE:

```
ai > ...
1 import os
2 import uuid
3 import logging
4 from pathlib import Path
5 from flask import Flask, request, abort, jsonify
6 from werkzeug.utils import secure_filename
7 from werkzeug.exceptions import HTTPException
8
9 # -----
10 # Configuration
11 # -----
12 BASE_DIR = Path(__file__).resolve().parent
13 # In production, keep uploads outside any static / served folder.
14 # This demo uses a subfolder, but production setups should use
15 # dedicated storage (e.g., S3 or a directory not served by the webserver).
16 UPLOAD_FOLDER = BASE_DIR / "uploads"
17 ALLOWED_EXTENSIONS = {".txt", ".pdf", ".png", ".jpg", ".jpeg", ".gif"} # whitelist
18 MAX_CONTENT_LENGTH = 8 * 1024 * 1024 # 8 MB max upload size
19
20 app = Flask(__name__)
21 app.config["UPLOAD_FOLDER"] = str(UPLOAD_FOLDER)
22 app.config["MAX_CONTENT_LENGTH"] = MAX_CONTENT_LENGTH
23
24 # Ensure the upload folder exists with safe permissions
25 UPLOAD_FOLDER.mkdir(parents=True, exist_ok=True)
26 # Set restrictive permissions (owner read/write only)
27 try:
28     os.chmod(UPLOAD_FOLDER, 0o700)
29 except Exception:
30     # chmod may fail on some OSes; ignore but log
```

```
31     pass
32
33 # -----
34 # Logging setup
35 # -----
36 logging.basicConfig(level=logging.INFO)
37 logger = logging.getLogger(__name__)
38
39 # -----
40 # Helper functions
41 # -----
42 def allowed_file_extension(filename: str) -> bool:
43     """Check file extension against allowed whitelist."""
44     _, ext = os.path.splitext(filename.lower())
45     return ext in ALLOWED_EXTENSIONS
46
47
48 def safe_save_file(uploaded_file) -> str:
49     """
50     Saves file using a secure filename + random prefix (UUID) and validates path.
51     Returns the saved file name (not full path).
52     Raises werkzeug.exceptions to produce appropriate HTTP errors.
53     """
54     # 1) Use werkzeug's secure_filename to remove dangerous characters
55     original_filename = uploaded_file.filename
56     safe_name = secure_filename(original_filename)
57     if not safe_name:
58         abort(400, "Invalid filename.")
```

```

61     if not allowed_file_extension(safe_name):
62         abort(400, "File type not allowed.")
63
64     # 3) Generate random filename to avoid collisions and leaks
65     unique_name = f"{uuid.uuid4().hex}_{safe_name}"
66
67     # 4) Resolve final path and ensure it's inside UPLOAD_FOLDER
68     final_path = Path(app.config["UPLOAD_FOLDER"]) / unique_name
69     try:
70         final_resolved = final_path.resolve(strict=False)
71         upload_dir_resolved = Path(app.config["UPLOAD_FOLDER"]).resolve(strict=True)
72     except Exception:
73         abort(500, "Server file system error.")
74
75     if upload_dir_resolved not in final_resolved.parents and final_resolved != upload_dir_resolved:
76         # final path is not inside upload dir -> possible path traversal
77         abort(400, "Invalid file path.")
78
79     # 5) Save file securely
80     uploaded_file.save(str(final_resolved))
81
82     # 6) Set safe file permissions (owner read/write)
83     try:
84         os.chmod(final_resolved, 0o600)
85     except Exception:
86         pass
87
88     return unique_name

```

```

91 # Flask Routes
92 # -----
93 @app.route("/", methods=["GET"])
94 def index():
95     return """<!doctype html>
96     <title>Secure File Upload</title>
97     <h1>Upload a File</h1>
98     <form method="post" action="/upload" enctype="multipart/form-data">
99         <input type="file" name="file">
100         <input type="submit" value="Upload">
101     </form>
102     """
103
104 @app.route("/upload", methods=["POST"])
105 def upload_file():
106     # 1) Check that the request actually contains a file part
107     if "file" not in request.files:
108         return jsonify({"error": "No file part in the request."}), 400
109
110     file = request.files["file"]
111
112     # 2) Basic checks
113     if file.filename == "":
114         return jsonify({"error": "No file selected."}), 400
115
116     # 3) Basic MIME check (not foolproof)
117     mimetype = file.mimetype or ""
118     if not any(mimetype.startswith(mt) for mt in ("image/", "application/", "text/")):
119         return jsonify({"error": "Unsupported media type."}), 400

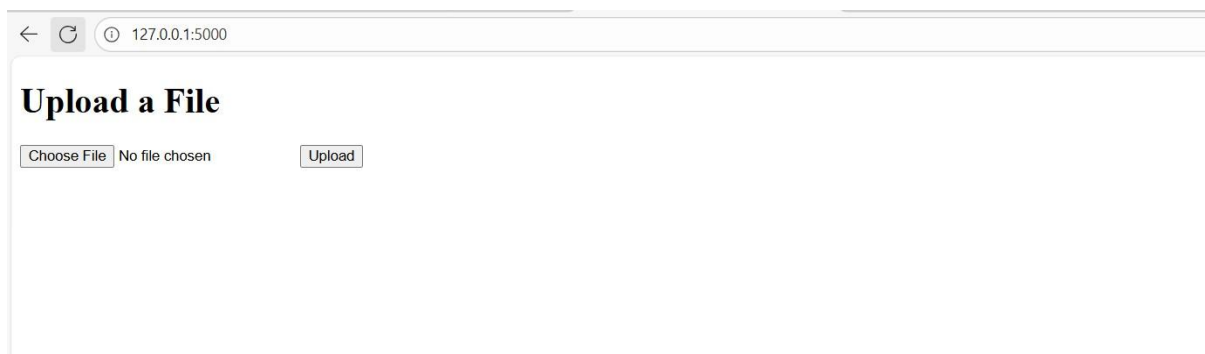
```

```

119         return jsonify({"error": "Unsupported media type."}), 400
120
121     # 4) Save file securely
122     try:
123         saved_name = safe_save_file(file)
124     except HTTPException:
125         raise # Re-raise HTTP errors for Flask to handle
126     except Exception as exc:
127         logger.exception("Failed to save upload: %s", exc)
128         abort(500, "Failed to save file.")
129
130     # 5) Log safely
131     logger.info("saved uploaded file as %s", saved_name)
132
133     # 6) Respond safely (do not expose paths)
134     return jsonify({"message": "File uploaded successfully.", "filename": saved_name}), 201
135
136 # -----
137 # Run (Development only)
138 # -----
139 if __name__ == "__main__":
140     # In production:
141     # - Run behind a WSGI server (gunicorn/uvicorn)
142     # - Do NOT use debug=True
143     app.run(host="127.0.0.1", port=5000, debug=False)
144

```

OUTPUT:



OBSERVATION:

1. Vulnerability: Unrestricted File Types & Remote

Code Execution (RCE): The script does not check file extensions. An attacker could upload a web shell (e.g., `shell.php`, `shell.aspx`) or an HTML file with malicious JavaScript (`phish.html`). If the `uploads` directory is ever served publicly, executing these files could lead to complete server compromise or XSS attacks on other users.

2. Vulnerability: Path Traversal: While

`secure_filename` is a good first step, it's not a complete defense. A sophisticated attacker might find ways to bypass it. The core issue is trusting user input to construct a file path. A payload like `../../tmp/evil.sh` could, in a misconfigured environment, write files outside the intended directory.

3. Vulnerability: Denial of Service (DoS): The script

does not enforce a file size limit (`MAX_CONTENT_LENGTH`). An attacker could upload a multi-gigabyte file, filling the server's disk space and causing the application or the entire server to crash.

4. Vulnerability: Filename Collision: If two users upload

a file named `resume.pdf`, the second upload will overwrite the first. This leads to data loss.

